

Quality Gates Before Implementation

Catch problems while the artifacts are still cheap to change, not after code exists.

What You'll Learn

1

Generate and use a domain-specific requirements-quality checklist

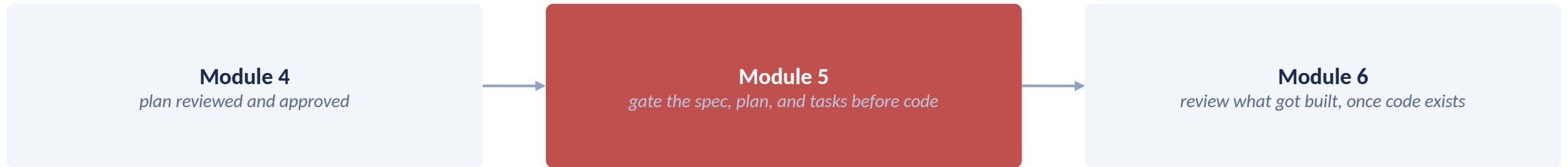
2

Run a cross-artifact consistency analysis before code is written

3

Budget agent token/cost usage and place security, compliance, and coverage checks as gates

A Reviewed Plan Can Still Hide a Broken Spec



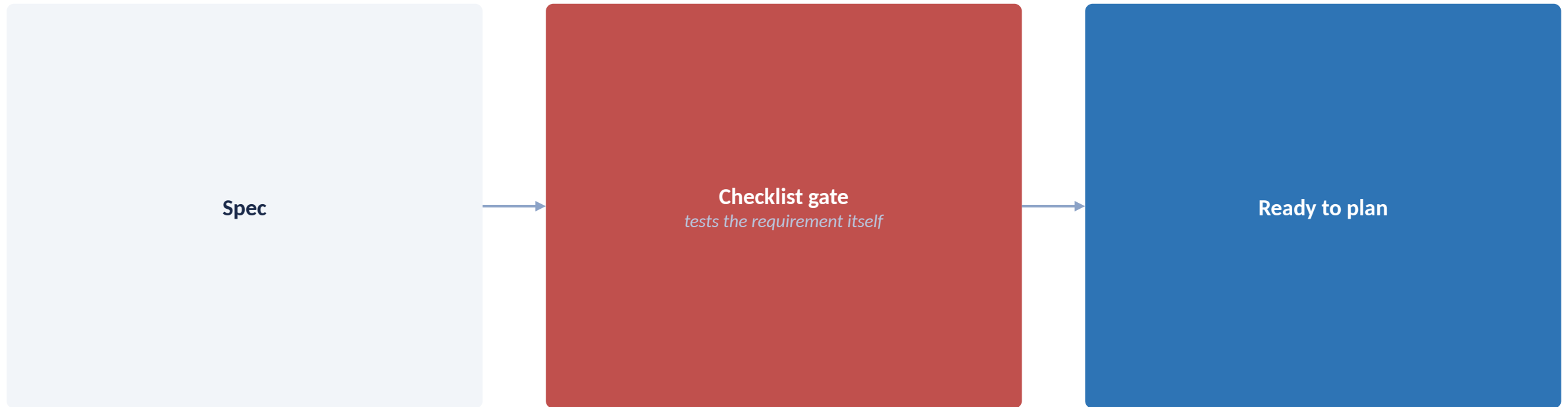
A plan can be internally consistent and still rest on an incomplete or ambiguous spec. Module 5 adds the checks that catch that before an agent starts writing code.

PART 1 • GATE ONE

Checklist: A Unit Test Suite for Requirements

Testing the requirement itself, before anything is built against it

Checklist: "Unit Tests for Requirements"



Unit tests for requirements, run before any code exists.

Generating a Custom Checklist

```
/speckit.checklist
```

Generates a custom, reviewer-owned quality checklist for the feature. Run broadly across the whole spec, or focused on one area, so a high-risk feature can get a deeper pass than a routine one without extra manual setup.

What it checks

Completeness (are all cases covered), clarity (can two readers interpret a requirement the same way), and consistency (do requirements contradict each other) — the same three properties EARS phrasing sets requirements up to satisfy.

Who owns it

The checklist is generated, but a human reviewer works through it and decides pass/fail per item. The tool surfaces gaps; it does not close them on its own.

Checkpoint: What does a checklist gate actually validate?

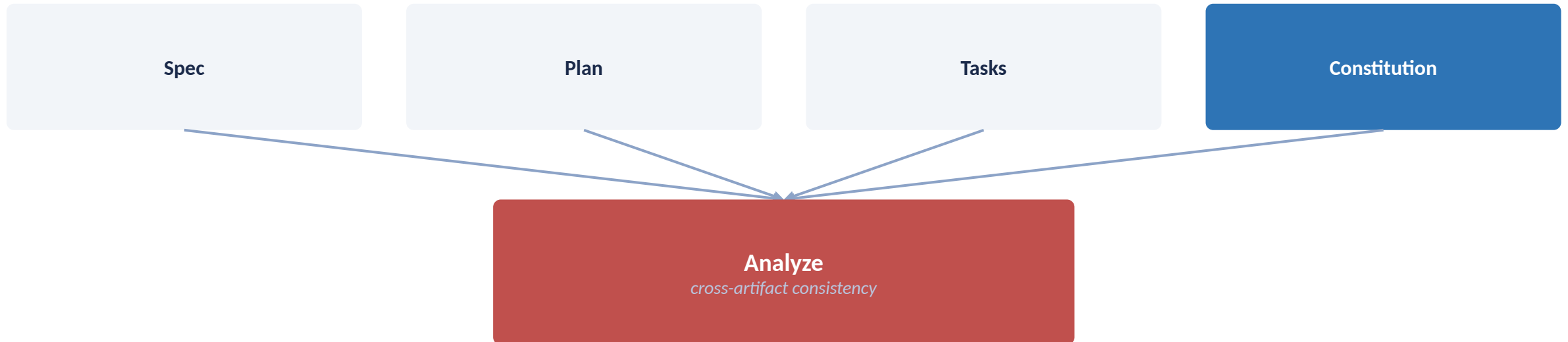
- The requirement itself, not the code, since no code exists yet at this point in the loop.
- Three properties: completeness, clarity, and consistency.
- A generated checklist still needs a human to walk through it and make the call.

PART 2 • GATE TWO

Analyze: Cross-Artifact Consistency

Catching drift between spec, plan, tasks, and constitution while it's still cheap to fix

Analyze: Cross-Artifact Consistency



Catches drift between artifacts while it's still cheap to fix.

Module 5 at a Glance

	GitHub Spec Kit	OpenSpec
Pre-code quality gate	/speckit.checklist + /speckit.analyze	None (closest command runs after code)
Timing	Before implementation	/opsx:verify runs after /opsx:apply
Takeaway	Purpose-built pre-implementation gates	Gates the equivalent concern later, in Module 6

Checkpoint: Why is Analyze read-only?

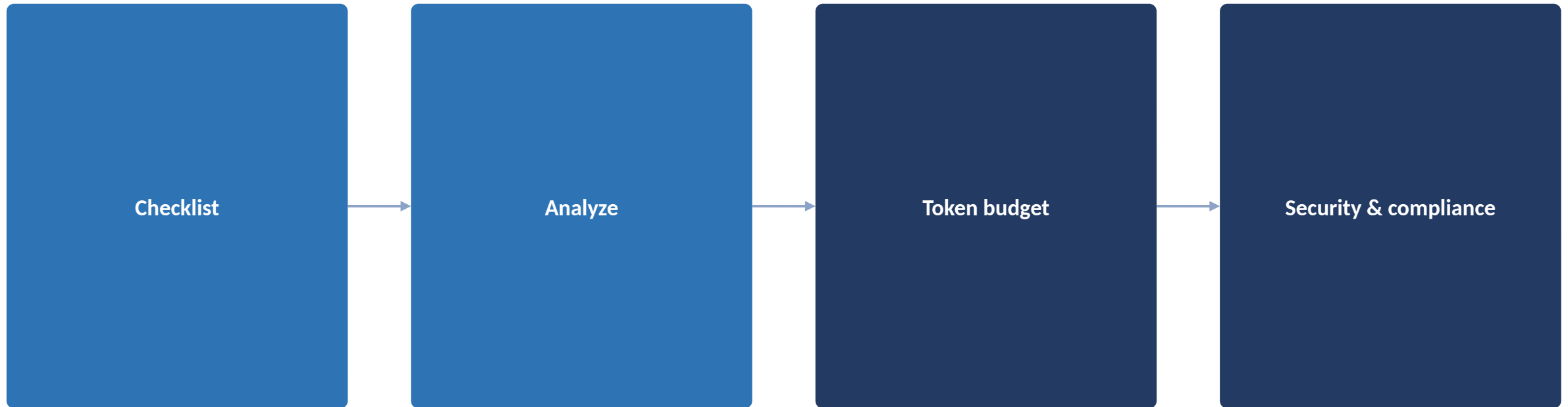
- So a false positive can't silently rewrite a spec, plan, or task file no one asked it to touch.
- It reports drift; a human decides which artifact is wrong and fixes it directly.
- The same discipline as plan review in Module 4: surface disagreement, don't auto-resolve it.

PART 3 • BEYOND THE GATES

Token Budget, Security, Compliance, and Coverage

Planned gates, not afterthoughts bolted on once something goes wrong

Four Gates, One Sequence



All four are planned gates, not afterthoughts bolted on later.

Budgeting Agent Token Usage and Cost

Why budget before implementation

Token cost scales with how much context an agent has to read and how many turns a task takes. A plan broken into oversized tasks (the Module 4 pitfall) doesn't just risk unreviewable diffs, it also runs up cost with no visibility until the invoice arrives.

What to estimate

Rough cost per task based on plan size and expected turns, a ceiling for the feature as a whole, and a checkpoint for what happens if a task blows past its estimate, pause and re-scope, not let it run.

Where it fits

Alongside checklist and analyze, before implementation starts, so token budget is a planning input, not a post-mortem line item.

Security, Compliance, and Coverage as Planned Gates

Security

Does the plan touch auth, data handling, or external calls in a way that needs a security review before implementation, not after? Flag it in the plan, not in a post-launch scan.

Compliance

If the feature touches regulated data or a documented policy (data retention, access logging, audit trail requirements), name the specific policy in the plan so the constitution and the plan agree on it.

Code coverage

Decide the coverage bar for this feature during planning, not as a CI gate discovered for the first time at merge. A plan that names its test strategy up front rarely fails this gate as a surprise.

Checkpoint: What do all four gates have in common?

- Every one of them runs before code exists, while the artifact under review is still cheap to change.
- Each is a planned, named check, not something discovered mid-implementation or after launch.
- None of them replace human judgment; they surface what needs a decision.

Common Pitfalls

Running checklist and analyze as a formality

Generating a checklist and rubber-stamping every item defeats the point. Someone has to genuinely evaluate each requirement against it.

Treating token budget as an afterthought

Estimating cost only after a task blows past budget means the estimate arrives too late to change anything.

Deferring security and compliance to a later review cycle

Waiting for a separate security pass after implementation turns a planning conversation into a rework cycle.

Key Takeaways

- Quality gates exist to catch problems while they're still cheap to fix, which is always before code, not after.
- Spec Kit has purpose-built pre-implementation gates (checklist, analyze); OpenSpec's equivalent, /opsx:verify, operates later in the loop, once there's code to check.
- Token budget, security, compliance, and coverage are planned gates alongside checklist and analyze, not items tacked on afterward.
- Module 6 picks up right where OpenSpec's /opsx:verify lives: reviewing what got built, once code exists.